

Lab 6

Programing with Python Lab

BCACC393

Recursion

1. Factorial Using Recursion

Write a Python program to define a recursive function `factorial(n)` that calculates the factorial of a given positive integer.

2. Fibonacci Series Using Recursion

Write a recursive function `fibonacci(n)` to generate the first n terms of the Fibonacci series.

3. Sum of Natural Numbers Using Recursion

Write a recursive function `sum_n(n)` that calculates the sum of all natural numbers from 1 to n .

4. Reverse a Number Using Recursion

Write a recursive function to reverse the digits of a given integer. For example, 12345 should produce 54321.

Closures

5. Simple Closure Function

Write a Python program that demonstrates a **closure** using an outer function `outer()` and an inner function `inner()`. The inner function should access a variable defined in the outer function.

6. Closure for Power Calculation

Create a function `power(n)` that returns another function. The returned function should calculate the power of n for a given value. Demonstrate the closure with different values.

7. Counter Using Closure

Create a closure-based counter function that maintains its count between function calls. The counter should increase by 1 every time the returned function is called.

Decorators

8. Basic Function Decorator

Write a decorator `display_message` that displays "Function execution started" before calling a function and "Function execution completed" after the function finishes.

9. Decorator for Measuring Execution

Create a decorator that records and displays the execution time of a function. Apply it to a function that performs a computational task.

10. Decorator with Arguments

Write a decorator that accepts a function and displays the arguments passed to that function before executing it.

Classes and Objects

11. Create a Class and Objects

Create a class `Student` with attributes `name`, `roll_no`, and `marks`. Create three objects of the class and display their details.

12. Class Attributes

Create a class `Employee` with a class attribute `company = "ABC Technologies"` and instance attributes `name` and `salary`. Create multiple objects and demonstrate the difference between class and instance attributes.

13. Methods in a Class

Create a class `Rectangle` with attributes `length` and `width`. Define methods to calculate and display the area and perimeter of the rectangle.

14. Private Methods

Create a class `BankAccount` with a private method `__calculate_interest()` that calculates interest on the account balance. Create a public method that calls the private method and displays the calculated interest.

Constructors and Destructors

15. Constructor in Python

Create a class `Book` with a constructor that initializes `title`, `author`, and `price`. Create an object and display its details.

16. Constructor and Destructor

Create a class `Demo` with a constructor that displays a message when an object is created and a destructor that displays a message when the object is destroyed. Create and delete an object explicitly.

Inheritance

17. Single Inheritance

Create a base class `Person` containing attributes `name` and `age`. Create a derived class `Student` that additionally contains `roll_no` and `course`. Display all details.

18. Multilevel Inheritance

Create three classes `Vehicle`, `Car`, and `ElectricCar` demonstrating **multilevel inheritance**. Each class should contain at least one attribute or method of its own.

19. Multiple Inheritance

Create two classes `Father` and `Mother`, each having a method displaying their respective skills. Create a class `Child` that inherits from both classes and demonstrate multiple inheritance.

Overloading and Method Overriding

20. Method Overloading Using Default Arguments

Python does not support traditional method overloading like C++ or Java. Demonstrate how **default arguments** can be used to achieve similar behavior by creating an `add()` method that can add two or three numbers.

21. Operator Overloading

Create a class `Point` with attributes `x` and `y`. Overload the `+` operator using `__add__()` to add two `Point` objects.

22. Method Overriding

Create a base class `Animal` with a method `sound()`. Create derived classes `Dog` and `Cat` that override the `sound()` method with their respective sounds.

Abstract Base Class

23. Abstract Base Class (ABC)

Create an abstract class `Shape` containing an abstract method `area()`. Create derived classes `Circle` and `Rectangle` and implement the `area()` method in both classes.

Metaclass

24. Basic Metaclass

Create a custom metaclass `MyMeta` that prints a message whenever a class using this metaclass is created. Create a class `Student` using `MyMeta` and demonstrate the behavior.

25. Metaclass for Automatic Class Modification

Create a metaclass that automatically adds a class attribute `category = "Python Class"` to every class created using that metaclass. Create two classes using the metaclass and display the automatically added attribute.